

SMART AUTOMATION SYSTEM

Cloud-Connected IoT System Using ESP32, LDR Sensor and Thingspeak

Internship Domain: Embedded Systems & IoT

Project Type: IoT Cloud Monitoring & Automation

Submitted By: Saurav Joshi

Course: B.Tech – Electronics and Communication Engineering

Institute: Bipin Tripathi Kumaon Institute of Technology (BTKIT), Dwarahat

Cloud Platform: ThingSpeak

Simulator: Wokwi ESP32 Simulator

ThingSpeak Channel ID: 3468746

CERTIFICATE

This is to certify that Saurav Joshi, a student of B.Tech in Electronics and Communication Engineering, has successfully completed Task-4 titled “Smart Automation System with ESP32 and ThingSpeak” as part of practical work in Embedded Systems and IoT. The project demonstrates sensor interfacing, automatic control, Wi-Fi communication, cloud data transmission, and real-time visualization using the ThingSpeak IoT platform. The work presented in this report demonstrates practical understanding of ESP32-based IoT systems and cloud-connected automation.

DECLARATION

I hereby declare that this project report presents the work completed by me for Task-4: Smart Automation System. The system was implemented and tested using the Wokwi ESP32 simulator. The ESP32 reads the LDR sensor, controls an LED automatically according to the detected light level, connects to Wi-Fi, and sends sensor data to the ThingSpeak cloud platform.

The report has been prepared for academic and internship documentation purposes.

Name: Saurav Joshi

Date: 28/08/2026

1. INTRODUCTION

The Internet of Things (IoT) is a technology in which physical devices are connected to the internet so that they can collect, transmit, monitor and process data remotely.

In Task-3, a Smart Automation System was developed using an LDR sensor and an LED. The system automatically controlled the LED according to the surrounding light intensity.

In Task-4, the existing automation system was upgraded into a cloud-connected IoT system using an ESP32 microcontroller and the ThingSpeak cloud platform.

The ESP32 has built-in Wi-Fi functionality, which makes it suitable for IoT applications. The LDR sensor measures the surrounding light level, while the ESP32 processes the sensor reading and controls the LED.

At the same time, the ESP32 sends the sensor reading and LED status to ThingSpeak through an HTTP request. ThingSpeak stores the data and displays it on a web-based dashboard in the form of graphs.

Thus, the project demonstrates the complete IoT workflow:

Sensor → ESP32 → Wi-Fi → Cloud → Dashboard

2. OBJECTIVE

The main objectives of this project are:

To upgrade the Task-3 Smart Automation System into an IoT-enabled system.

To use ESP32 for sensor processing and Wi-Fi connectivity.

To interface an LDR/photoresistor with ESP32.

To automatically control an LED according to surrounding light conditions.

To connect the ESP32 to a Wi-Fi network.

To send real-time sensor readings to ThingSpeak.

To display sensor readings on a cloud dashboard.

To monitor the changing sensor values remotely.

To understand device-to-cloud communication.

To gain practical experience in Embedded Systems and IoT.

3. PROJECT OUTCOME

The completed project provides a cloud-connected Smart Automation System.

The ESP32 continuously reads the LDR sensor and automatically controls the LED. The sensor data is transmitted through Wi-Fi to ThingSpeak.

The ThingSpeak dashboard displays the received data in real time through graphical charts.

The project therefore combines:

Embedded system

Sensor interfacing

Automatic control

Wi-Fi connectivity

Cloud communication

Data visualization

4. HARDWARE AND SOFTWARE REQUIREMENTS

4.1 Hardware Components

Component	Purpose
ESP32 Devkit	Main microcontroller and Wi-Fi communication
LDR/Photoresistor	Detects surrounding light
LED	Indicates automatic lighting condition
220 Ω Resistor	Limits LED current
Jumper Wires	Electrical connections

4.2 Software/Platforms

Software/Platform	purpose
Wokwi	ESP32 circuit simulation
Arduino/C++	Programming ESP32
ThingSpeak	Cloud data storage and visualization
Wi-Fi	Internet connectivity
HTTP	Data transmission to cloud

5. SYSTEM ARCHITECTURE

The overall architecture of the system is:

LDR Sensor → ESP32 → Wi-Fi → ThingSpeak Cloud → Web Dashboard

Working Flow

The LDR detects the surrounding light level.

The LDR provides an analog signal to the ESP32.

ESP32 reads the sensor value through GPIO 34.

The sensor value is compared with a predefined threshold.

The LED is automatically turned ON or OFF.

ESP32 connects to the Wokwi-GUEST Wi-Fi network.

ESP32 sends the sensor reading to ThingSpeak.

ThingSpeak stores the received data.

The data is displayed on the ThingSpeak dashboard.

6. PIN CONNECTIONS

Component	Pin	ESP32 Connection
Photoresistor	VCC	3.3V
Photoresistor	GND	GND
Photoresistor	AO	GPIO 34
Photoresistor	DO	Not Connected
LED	Anode (+)	GPIO 2
LED	Cathode (-)	220 Ω Resistor → GND

7. CIRCUIT IMPLEMENTATION

The circuit was developed using the Wokwi ESP32 simulator.

The main components used are:

- ESP32 DevKit

- Photoresistor/LDR
- LED
- 220 Ω resistor
- Jumper connections

The analog output of the photoresistor is connected to GPIO 34 of the ESP32.

The LED is connected to GPIO 2 through a current-limiting resistor.

Circuit Screenshot

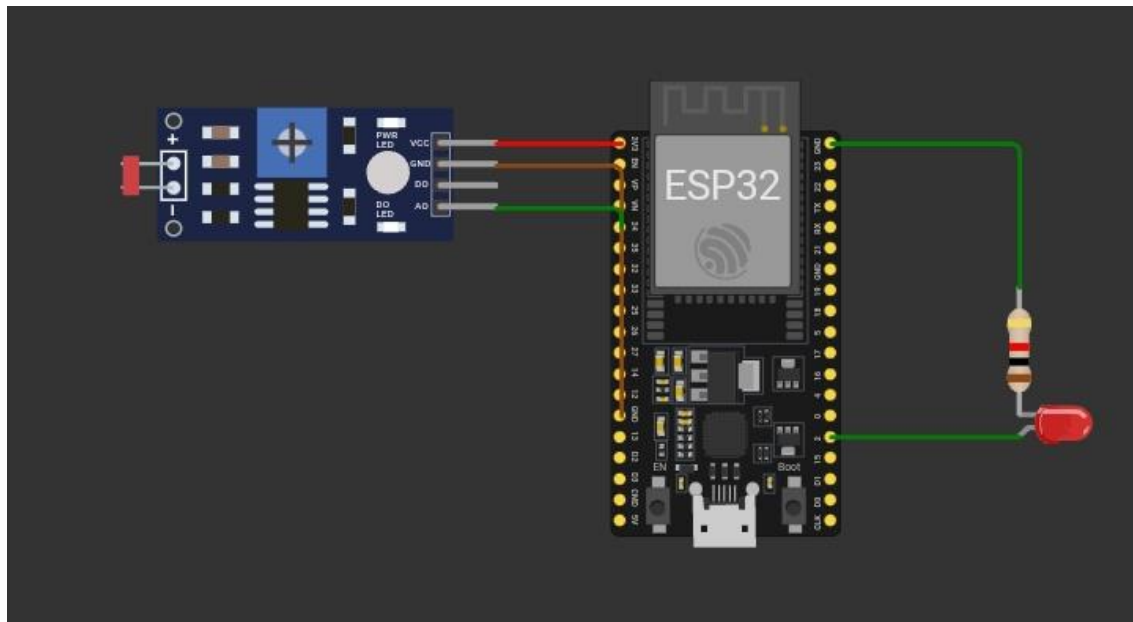


Figure 1: ESP32-based Smart Automation System circuit in Wokwi

8. WORKING PRINCIPLE

The working principle of the system is based on automatic light detection.

Step 1 – Light Detection

The LDR/photoresistor detects the intensity of surrounding light.

Step 2 – Sensor Reading

ESP32 reads the analog value from the LDR using:

```
analogRead(ldrPin);
```

The sensor is connected to GPIO 34.

Step 3 – Automatic LED Control

The sensor value is compared with a threshold value.

The implemented logic is:

Low Light → LED ON

Bright Light → LED OFF

Step 4 – Wi-Fi Connection

The ESP32 connects to the Wokwi-GUEST Wi-Fi network.

SSID: Wokwi-GUEST

Password: Empty

Step 5 – Cloud Communication

After connecting to Wi-Fi, the ESP32 sends the sensor reading to ThingSpeak using an HTTP GET request.

Step 6 – Cloud Dashboard

ThingSpeak receives the data and displays the values through charts.

9. SOURCE CODE

Important: Report/GitHub me apni real ThingSpeak Write API Key public mat karna. Neeche placeholder use kiya gaya hai.

```
#include <WiFi.h>
```

```
#include <HTTPClient.h>
```

```
// Pin Configuration
```

```
Const int ldrPin = 34;
```

```
Const int ledPin = 2;
```

```

// Wokwi Wi-Fi

Const char* ssid = "Wokwi-GUEST";
Const char* password = "";

// ThingSpeak Configuration
Const char* apiKey = "YOUR_WRITE_API_KEY";
Const char* server = http://api.thingspeak.com/update;

Void setup() {

    Serial.begin(115200);

    pinMode(ldrPin, INPUT);
    pinMode(ledPin, OUTPUT);

    // Connect ESP32 to Wi-Fi
    WiFi.begin(ssid, password);

    Serial.print("Connecting to WiFi");

    While (WiFi.status() != WL_CONNECTED) {

        Delay(500);
        Serial.print(".");
    }

    Serial.println();
    Serial.println("WiFi Connected!");
}

```

```
Serial.print("IP Address: ");  
Serial.println(WiFi.localIP());  
  
Serial.println("Smart Automation System Started");  
}
```

```
Void loop() {
```

```
    // Read LDR sensor value
```

```
    Int lightValue = analogRead(ldrPin);
```

```
    // Low light = LED ON
```

```
    // Bright light = LED OFF
```

```
    If (lightValue > 500) {
```

```
        digitalWrite(ledPin, HIGH);
```

```
        Serial.println("Low Light – LED ON");
```

```
    }
```

```
    Else {
```

```
        digitalWrite(ledPin, LOW);
```

```
        Serial.println("Enough Light – LED OFF");
```

```
    }
```

```
    // Display sensor value
```

```

Serial.print("Light Value: ");
Serial.println(lightValue);

// Send data to ThingSpeak

If (WiFi.status() == WL_CONNECTED) {

    HTTPClient http;

    // 1 = LED ON
    // 0 = LED OFF

    Int ledStatus = (lightValue > 500) ? 1 : 0;

    String url = String(server) +
        "?api_key=" + apiKey +
        "&field1=" + String(lightValue) +
        "&field2=" + String(ledStatus);

    http.begin(url);

    int httpCode = http.GET();

    Serial.print("ThingSpeak Response: ");
    Serial.println(httpCode);

    http.end();
}

```

```
// ThingSpeak requires delay between updates  
Delay(16000);  
}
```

10. CODE EXPLANATION

Wi-Fi Library

```
#include <WiFi.h>
```

This library allows the ESP32 to connect to a Wi-Fi network.

HTTP Library

```
#include <HTTPClient.h>
```

This library is used to send HTTP requests to ThingSpeak.

Sensor Pin

```
Const int ldrPin = 34;
```

GPIO 34 is used to read the analog LDR value.

LED Pin

```
Const int ledPin = 2;
```

GPIO 2 controls the LED.

Wi-Fi Connection

```
WiFi.begin(ssid, password);
```

This connects the ESP32 to the configured Wi-Fi network.

Sensor Reading

```
Int lightValue = analogRead(ldrPin);
```

This reads the current LDR sensor value.

Automatic Control

```
If (lightValue > 500)
```

The program uses a threshold of 500 for automatic LED control.

ThingSpeak Data

The following values are sent to ThingSpeak:

Field 1 → Light Sensor Value

Field 2 → LED Status

LED status:

1 → LED ON

0 → LED OFF

HTTP Response

ThingSpeak Response: 200

A response code of 200 indicates that the HTTP request was successfully processed.

11. CLOUD INTEGRATION WITH THINGSPEAK

For cloud connectivity, the ThingSpeak IoT cloud platform was selected.

A channel named:

Smart Automation System

Was created.

Channel Details

Channel ID: 3468746

Fields Used

Field	Data
Field 1	Light Intensity
Field 2	LED Status

Field 1 stores the LDR/ADC sensor value.

Field 2 stores the LED status.

1 = LED ON

0 = LED OFF

The ESP32 sends the data to ThingSpeak using an HTTP GET request.

12. THINGSPEAK DASHBOARD

The ThingSpeak dashboard provides graphical visualization of the sensor data received from ESP32.

The dashboard shows changing values as new readings are uploaded.

Dashboard Screenshot

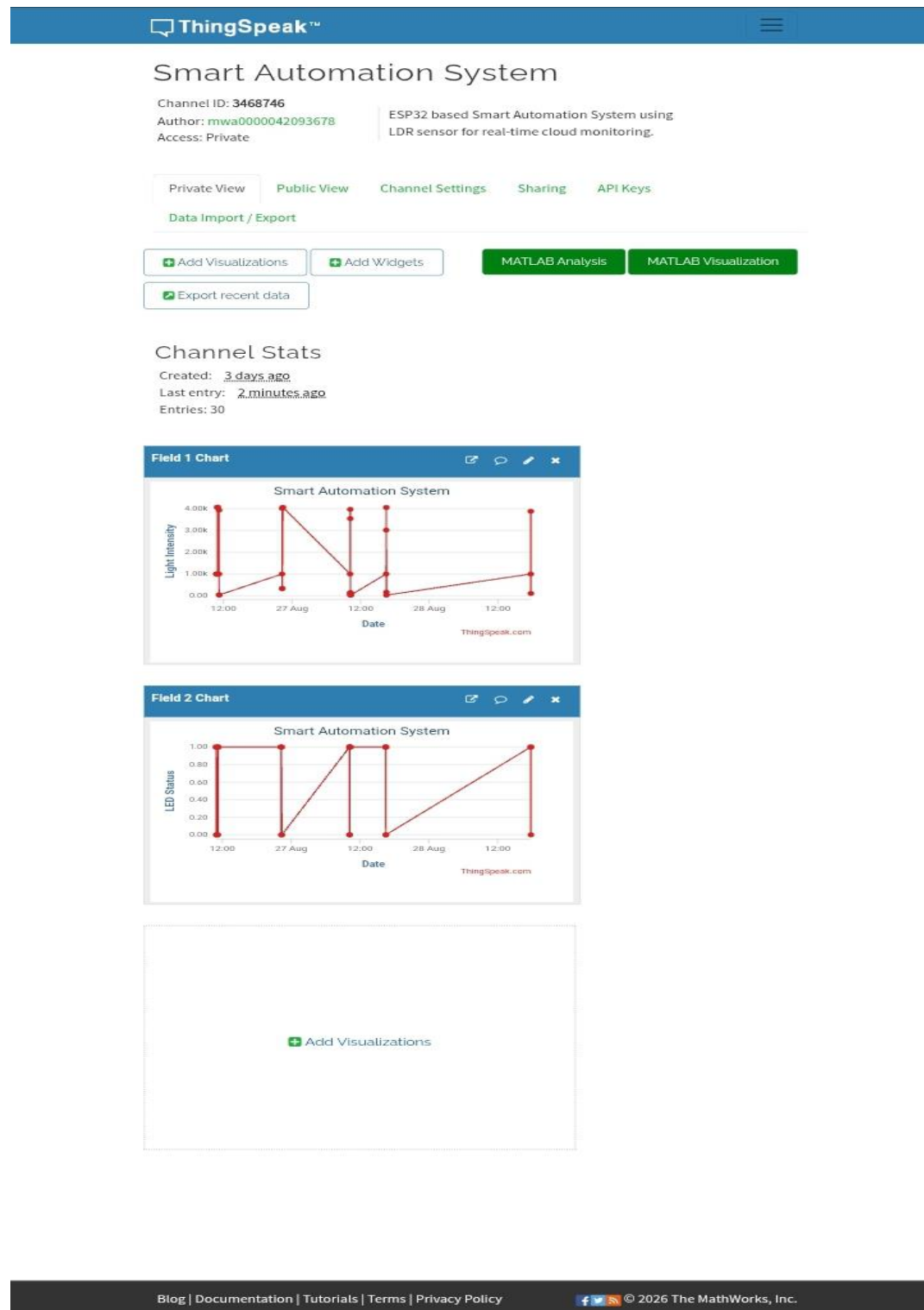


Figure 2: ThingSpeak dashboard showing real-time sensor data

The graph demonstrates that the ESP32 is successfully sending changing sensor readings to the cloud.

13. DATA TRANSMISSION PROCESS

14. The complete cloud communication process is:

LDR Sensor



ESP32 Reads Sensor



Sensor Value Processing



LED Automatic Control



ESP32 Wi-Fi Connection



HTTP GET Request



ThingSpeak Cloud



Channel Data Storage



Dashboard Graph

This demonstrates a complete Device-to-Cloud IoT communication system.

15. TESTING AND RESULTS

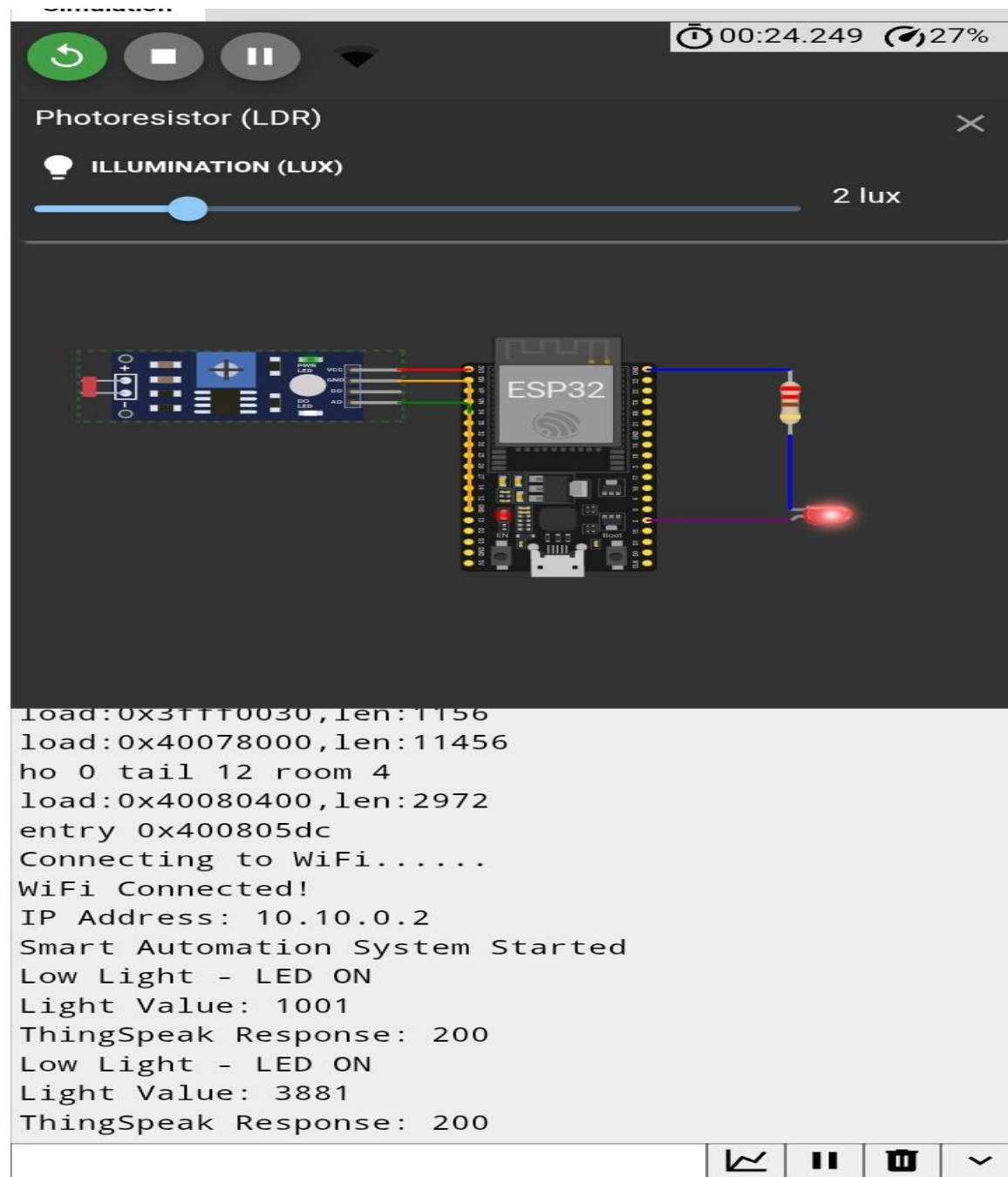
The system was tested under different light conditions.

Condition	Expected Result	Observed Result
Low Light.	LED ON	LED ON
Bright Light	LED OFF	LED OFF

Wi-Fi Available	ESP32 connects	Connected
Cloud Communication	Data uploaded	Successfully uploaded
HTTP Request	Successful response	Response 200

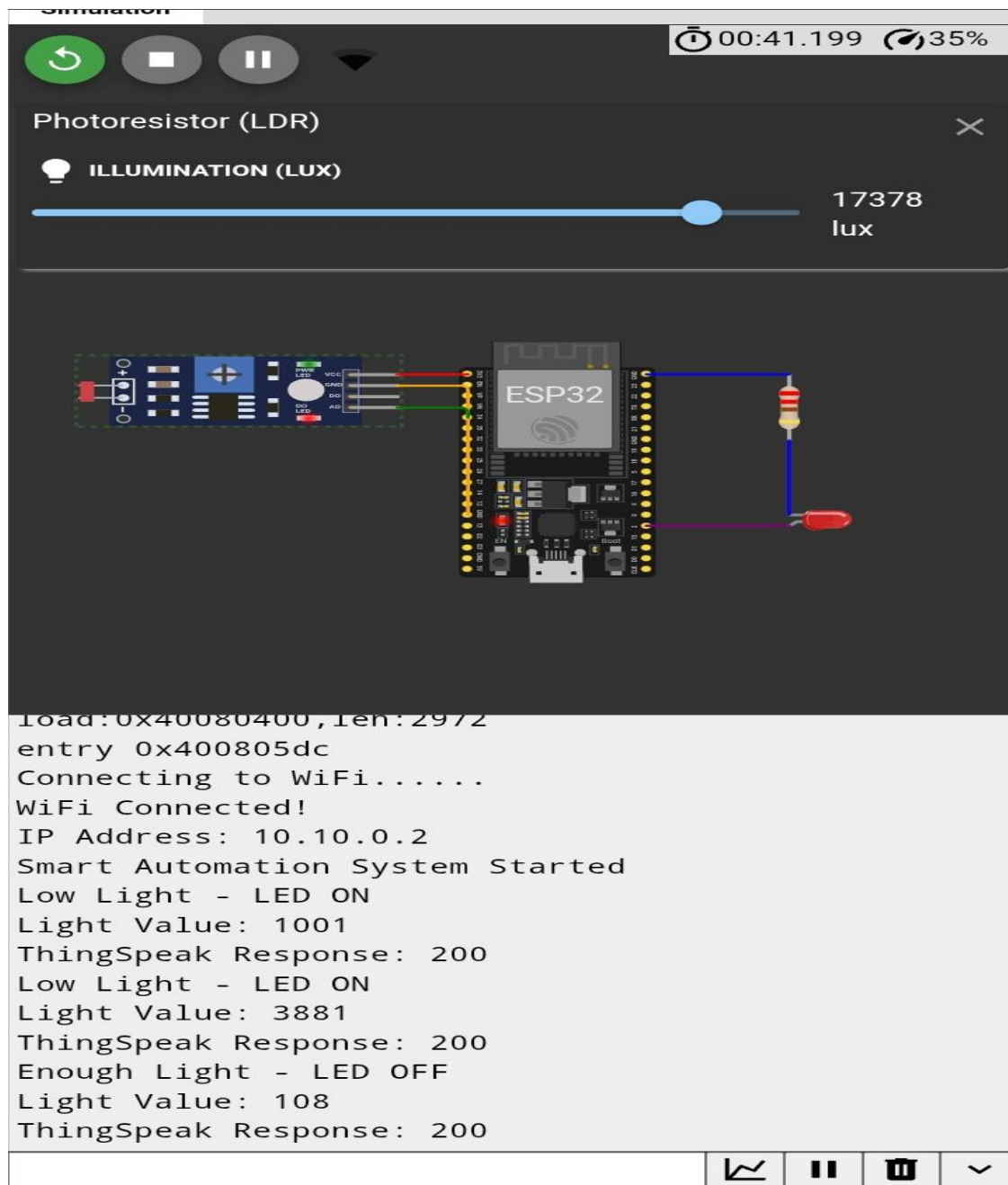
Test 1 – Low Light

When the light intensity is reduced, the system detects the low-light condition and turns the LED ON.



Test 2 – Bright Light

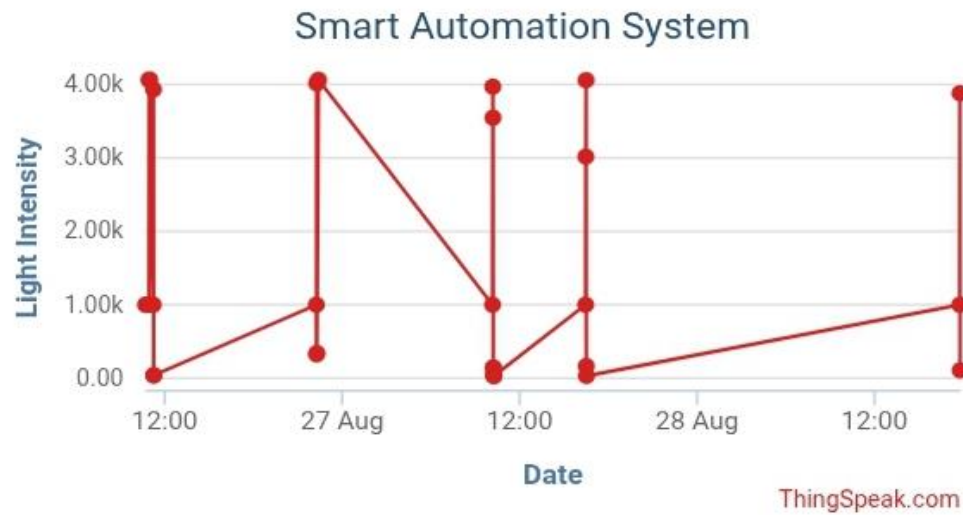
When the light intensity is increased, the system detects sufficient light and turns the LED OFF.



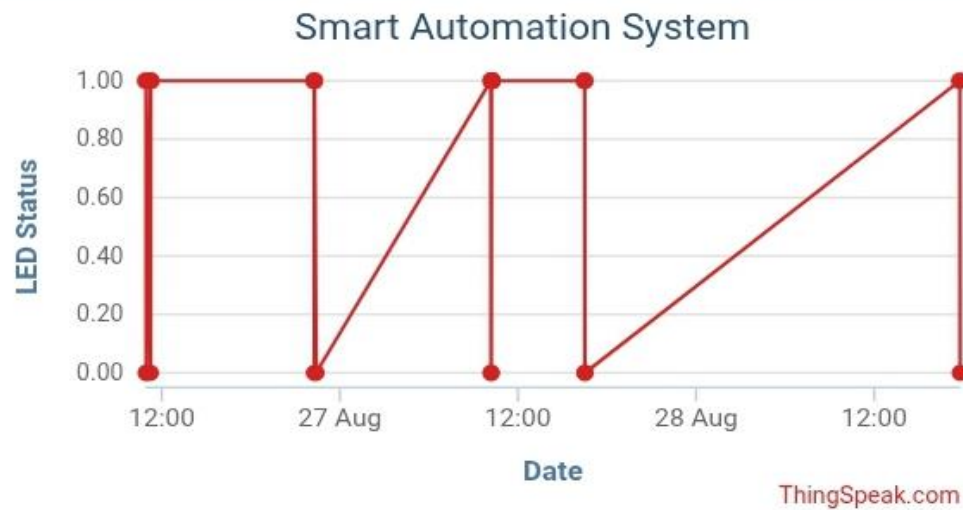
Test 3 – Cloud Data

The corresponding sensor readings are visible on the ThingSpeak graph.

Field 1 Chart



Field 2 Chart



16. LREAL-TIME MONITORING

One of the main improvements in Task-4 is real-time cloud monitoring.

In Task-3, the system could only demonstrate automatic control locally.

In Task-4, the sensor data is transmitted to the internet.

Therefore, the system now provides:

Local automatic control

Wi-Fi connectivity

Cloud data transmission

Remote monitoring

Graphical data visualization

This makes the project more similar to a practical IoT application.

17. APPLICATIONS

The developed system can be used in various applications:

Smart Home Lighting

Lights can automatically turn ON when the environment becomes dark.

Smart Classrooms

Classroom lighting can be controlled automatically according to ambient light.

Energy Saving

Automatic lighting can reduce unnecessary electricity consumption.

Smart Buildings

The system can be integrated into building automation systems.

IoT Monitoring

Sensor values can be monitored remotely using cloud platforms.

Industrial Automation

Similar architecture can be used for monitoring and controlling industrial systems.

18. ADVANTAGES

The major advantages of the system are:

Simple and low-cost design.
ESP32 provides built-in Wi-Fi.
Automatic LED control.
Cloud-based data monitoring.
Real-time graphical visualization.
Easy to expand with additional sensors.
Suitable for IoT learning and prototyping.
Can be converted from simulation to real hardware.

19. LIMITATIONS

The current system also has some limitations.
The project is currently implemented using a simulator.
The LDR value is used as a relative sensor/ADC reading rather than a calibrated physical lux measurement.
The system depends on Wi-Fi connectivity for cloud monitoring.
ThingSpeak has update-rate limitations for free usage.
The current system contains only one main sensor.
For an actual hardware implementation, the system would require proper sensor calibration and physical ESP32 hardware.

20. FUTURE SCOPE

The project can be further improved by adding:
Temperature and humidity sensors.
PIR motion sensor.
Automatic fan control.
Multiple LEDs.
Mobile notifications.
Email alerts.
Configurable threshold values.

Multiple ThingSpeak fields.

Energy consumption monitoring.

Mobile IoT dashboard.

Real ESP32 hardware deployment.

AI-based automation and prediction.

The system can eventually be developed into a complete Smart Home Automation System.

21. CONCLUSION

Task-4 successfully upgraded the Task-3 Smart Automation System into a cloud-connected IoT system.

The ESP32 reads the LDR sensor, processes the sensor value and automatically controls the LED according to the surrounding light condition.

Using its built-in Wi-Fi capability, the ESP32 connects to the internet and sends the sensor data to the ThingSpeak cloud platform through an HTTP request.

ThingSpeak receives and stores the data and displays it through graphical charts, allowing the sensor readings and LED status to be monitored remotely.

The project demonstrates the complete IoT workflow:

Sensing → Processing → Automation → Wi-Fi → Cloud → Visualization

This task provided practical knowledge of ESP32, LDR interfacing, Wi-Fi communication, HTTP requests, cloud IoT platforms and real-time data visualization, making it a useful foundation for more advanced IoT and embedded systems projects.

22. REFERENCES

Wokwi ESP32 Simulation Platform

ThingSpeak IoT Cloud Platform

Arduino/C++ Programming Documentation

ESP32 Wi-Fi Library Documentation

ESP32 HTTP Client Library Documentation

Task-4 Embedded Systems & IoT Internship Instructions

